Week-3

April 25, 2026

1 Import the packages

[1]:
```
import pandas as pd
import numpy as np
from scipy.integrate import quad
from scipy.optimize import minimize
import matplotlib.pyplot as plt
```

2 Load the Dataset

[2]:
```
# Load the dataset
file_path = 'data/Pantheon+SH0ES.dat'
try:
    df = pd.read_csv(file_path, sep=r'\s+')
    print("Dataset loaded successfully.")

    # We will need the error column for the chi-square weights.
    # In Pantheon+SH0ES, the diagonal error is typically 'm_b_corr_err_DIAG' or␣
 ↪'mBERR'.
    # We'll set a standard variable name for our script to use:
    if 'm_b_corr_err_DIAG' in df.columns:
        df['error_m_b'] = df['m_b_corr_err_DIAG']
    elif 'mBERR' in df.columns:
        df['error_m_b'] = df['mBERR']
    else:
        # Fallback if specific error column is missing
        df['error_m_b'] = 0.1
        print("Warning: Specific error column not found. Using default constant␣
 ↪error.")

    display(df[['CID', 'zCMB', 'zHEL', 'm_b_corr', 'CEPH_DIST',␣
 ↪'IS_CALIBRATOR', 'error_m_b']].head())
except FileNotFoundError:
    print(f"Error: The file {file_path} was not found.")
```

```
Dataset loaded successfully.
```

```
             CID     zCMB     zHEL  m_b_corr  CEPH_DIST  IS_CALIBRATOR  \
0         2011fe  0.00122  0.00082   9.74571    29.1770              1
1         2011fe  0.00122  0.00082   9.80286    29.1770              1
2         2012cg  0.00256  0.00144  11.47030    30.8433              1
3         2012cg  0.00256  0.00144  11.49190    30.8433              1
4  1994DRichmond  0.00299  0.00187  11.52270    -9.0000              0

   error_m_b
0   1.516210
1   1.517230
2   0.781906
3   0.798612
4   0.880798
```

3 Build Model Prediction Functions

[3]:
```python
# Speed of light in km/s
c = 299792.458

# ------------------------------------------------------------
# Model prediction for Cepheid Calibrators
# ------------------------------------------------------------
def model_calibrators(M_B, mu_ceph):
    """
    Computes the theoretical apparent magnitude for calibrators.
    Formula: m_b,th = M_B + mu_Ceph
    """
    return M_B + mu_ceph

# ------------------------------------------------------------
# Model prediction for Cosmological Supernovae
# ------------------------------------------------------------
def E_inverse(z, Omega_m):
    """
    Inverse of the dimensionless Hubble parameter E(z) for a flat Universe.
    E(z) = sqrt(Omega_m * (1+z)^3 + Omega_Lambda)
    Assuming flat universe: Omega_Lambda = 1 - Omega_m
    """
    Omega_L = 1.0 - Omega_m
    return 1.0 / np.sqrt(Omega_m * (1 + z)**3 + Omega_L)

def luminosity_distance_mpc(z_hel, z_cmb, H_0, Omega_m):
    """
    Calculates the luminosity distance in Megaparsecs (Mpc).
    """
    # Integrate 1/E(z) from 0 to z_cmb
    integral, _ = quad(E_inverse, 0, z_cmb, args=(Omega_m,))
```

```
    # D_L formula in cosmology
    d_L = (c / H_0) * (1 + z_hel) * integral
    return d_L

def model_cosmology(z_hel, z_cmb, M_B, H_0, Omega_m):
    """
    Computes theoretical apparent magnitude for standard supernovae.
    Formula: m_b,th = M_B + mu_LCDM
    Where mu_LCDM = 5 * log10(D_L_in_pc) - 5 = 5 * log10(D_L_in_Mpc) + 25
    """
    d_L_mpc = luminosity_distance_mpc(z_hel, z_cmb, H_0, Omega_m)
    mu_lcdm = 5.0 * np.log10(d_L_mpc) + 25.0
    return M_B + mu_lcdm
```

4 Combine Residuals and Define Simplified Chi-Square

[4]:
```
# ------------------------------------------------------------
# Residual Vector & Simplified Chi-Square
# ------------------------------------------------------------
def simplified_chi_square(params, data):
    """
    Calculates the simplified chi-square for the dataset.
    params: [Omega_m, H_0, M_B]
    """
    Omega_m, H_0, M_B = params

    # Apply reasonable physical bounds to prevent math domain errors during␣
 ↪optimization
    if Omega_m <= 0 or Omega_m >= 1 or H_0 <= 0:
        return np.inf

    chi2 = 0.0

    # We iterate over rows to compute theoretical predictions
    # (Note: A vectorized approach is faster, but a loop with `quad` is safest␣
 ↪for integration)
    for index, row in data.iterrows():
        m_b_obs = row['m_b_corr']
        sigma = row['error_m_b']
        is_calib = row['IS_CALIBRATOR']

        if is_calib == 1:
            # Calibrator model
            mu_ceph = row['CEPH_DIST']
            if mu_ceph == -9 or mu_ceph == -99 or np.isnan(mu_ceph):
```

```
                continue # Skip if Cepheid distance is missing
            m_b_th = model_calibrators(M_B, mu_ceph)
        else:
            # Cosmological model
            m_b_th = model_cosmology(row['zHEL'], row['zCMB'], M_B, H_0,␣
↪Omega_m)

        # Add to chi-square summation
        chi2 += ((m_b_obs - m_b_th) ** 2) / (sigma ** 2)

    return chi2
```

5 Preliminary Best Fit

[5]:
```
print("Starting minimization process... (this may take a few moments)")

# Initial guesses for the parameters: [Omega_m, H_0, M_B]
# Standard cosmological priors: Omega_m ~ 0.3, H_0 ~ 70, M_B ~ -19.2
initial_guess = [0.3, 70.0, -19.2]

# Define bounds for the parameters: (min, max)
# Omega_m between 0.01 and 0.99, H_0 between 50 and 100, M_B between -21 and -18
bounds = ((0.01, 0.99), (50.0, 100.0), (-21.0, -18.0))

# Perform the minimization using the L-BFGS-B method (supports bounds)
result = minimize(
    simplified_chi_square,
    x0=initial_guess,
    args=(df,),
    method='L-BFGS-B',
    bounds=bounds
)

# Output the results
if result.success:
    print("\nMinimization Successful!")
    print(f"Preliminary Best-Fit Parameters:")
    print(f" - Matter Density (Omega_m) = {result.x[0]:.4f}")
    print(f" - Hubble Constant (H_0)    = {result.x[1]:.2f} km/s/Mpc")
    print(f" - Absolute Magnitude (M_B) = {result.x[2]:.4f}")
    print(f"\nMinimum Chi-Square Value: {result.fun:.2f}")
else:
    print("\nMinimization Failed.")
    print(result.message)
```

```
Starting minimization process… (this may take a few moments)
```

```
Minimization Successful!
Preliminary Best-Fit Parameters:
 - Matter Density (Omega_m) = 0.3756
 - Hubble Constant (H_0)    = 72.55 km/s/Mpc
 - Absolute Magnitude (M_B) = -19.2517

Minimum Chi-Square Value: 764.42
```